

fair-publish

Policy-enforced FAIR metadata CI/CD

Live demo — github.com/in-silico-institute

M.-A. Sicilia · A. Ballesteros-Rodríguez · E. García-Barriocanal · M.
Mora-Cantallops · A. Nogales

University of Alcalá | IRyCIS

CRIS2026



Universidad
de Alcalá

The demo organisation

Architecture

Live demo

Rule system

Adoption

Summary

.github (governance)

Org-level special repo.

.github/workflows/

- ▶ fair-validate.yml
- ▶ fair-publish.yml

.github/policies/

- ▶ fair-policy.yml

Owned by institution.

*Policies update all
project repos on next push.*

fair-publish (engine)

Generic, institution-agnostic
package. GHA steps call:

```
fair-publish validate  
fair-publish publish
```

Institution-specific rules
live in .github, not here.

*Installed at runtime via
pip install git+...*

demo-dataset (project)

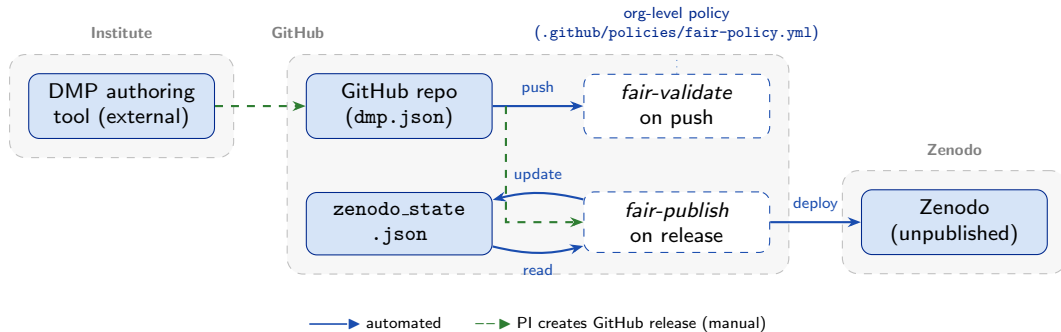
One PI's dataset repo.

- ▶ dmp.json — the maDMP
- ▶ zenodo_state.json
- ▶ .github/workflows/
 - validate.yml
 - publish.yml

*PI (or UI tool) pushes
dmp.json.*

Everything else is inherited.

The publication pipeline



Workflow 1 — `fair-validate.yml`

Trigger: *push / pull request (paths: dmp.json)*

- ▶ Structural validation via `madmpy`
- ▶ Runs each policy rule against every dataset
- ▶ Exit 1 if any error-level rule fails
- ▶ Writes rule-by-rule table to GHA step summary

Governance requirement

Branch protection must require this check — otherwise a PI can create a release without validation passing

Workflow 2 — `fair-publish.yml`

Trigger: *GitHub release published*

- ▶ Trusts the validation gate (no re-validation)
- ▶ New dataset → create new Zenodo record
- ▶ Known dataset → new version (via `zenodo_state.json`)
- ▶ Commits updated state file back to repo

Why split?

Validation gives fast feedback on every push. Publishing only happens at a deliberate version event. The gate cannot be bypassed.

Open: github.com/in-silico-institute/demo-dataset/blob/main/dmp.json

Listing 1: dmp.json (key fields)

```
{
  "dmp": {
    "title": "Longitudinal cohort study ...",
    "language": "eng",
    "contact": {
      "name": "Garcia, Elena",
      "contact_id": {
        "identifier":
          "https://orcid.org/0000-0001-6752-9599",
        "type": "orcid"
      }
    },
    "dataset": [{
      "title": "Metabolic biomarker dataset",
      "dataset_id": {
        "identifier":
          "metabolic-biomarker-cohort-wave1-5",
        "type": "other"
      }
    },
    "distribution": [{
      "data_access": "open",
      "license": [{
        "license_ref":
          "https://creativecommons.org/licenses/by/4.0/"
      }]
    }]
  }
}
```

What the pipeline reads:

- ▶ `contact.contact_id` → ORCID check
- ▶ `dataset[].keyword` → keyword check
- ▶ `distribution[].license` → license check
- ▶ `distribution[].data_access` → access rights
- ▶ `dataset_id.identifier` → state key
- ▶ `contributor[]` → Zenodo creators

Parsed by madmpy (RDA maDMP Common Standard).

Open: github.com/in-silico-institute/demo-dataset/actions

Select the **validate.yml** workflow — *most recent run*

What triggers it:

```
on:
  push:
    branches: [main]
    paths: ["dmp.json"]
  pull_request:
    paths: ["dmp.json"]
jobs:
  validate:
    uses: in-silico-institute/.github/
      .github/workflows/
      fair-validate.yml@main
    with:
      dmp_path: dmp.json
      policy_path:
        .github/policies/fair-policy.yml
```

Actual run output (step summary):

```
DMP: Longitudinal cohort study of metabolic biomarkers
Datasets: 1 Errors: 0 Warnings: 0
```

```
Dataset: Metabolic biomarker dataset [metabolic-biomarker-cohort-
wave1-5]
[F1, A1] Creator ORCID present OK
[R1.1] License present & recognized OK
[F2] At least one keyword OK
[F2, R1] Description length OK
[A1] Access rights stated OK
[F1] Dataset identifier present OK
```

```
PASSED
```

All 6 rules pass. Exit code 0.

A failing rule would exit 1 and block the release.

Open: `github.com/in-silico-institute/demo-dataset/actions`

Select the **publish.yml** workflow — *most recent successful run*

What triggers it:

```
on:
  release:
    types: [published]

jobs:
  publish:
    uses: in-silico-institute/.github/
      .github/workflows/
      fair-publish.yml@main
    with:
      dmp_path: dmp.json
      sandbox: true
      secrets: inherit
```

ZENODO_TOKEN stored as
org-level secret — PI never
handles credentials directly.

Actual run output:

```
DMP validated!

[created] Metabolic biomarker dataset
        cohort wave 1-5
        -> 10.5072/zenodo.499935

{
  "published": {
    "metabolic-biomarker-cohort-wave1-5":
      "10.5072/zenodo.499935"
  },
  "new": { "metabolic-...": "10.5072/zenodo.499935" },
  "updated": {}
}

[main] chore: update zenodo_state.json
```

Zenodo sandbox record created.
State file committed back automatically.

Open:

`github.com/in-silico-institute/demo-dataset/blob/main/zenodo_state.json`

What's in the repo now:

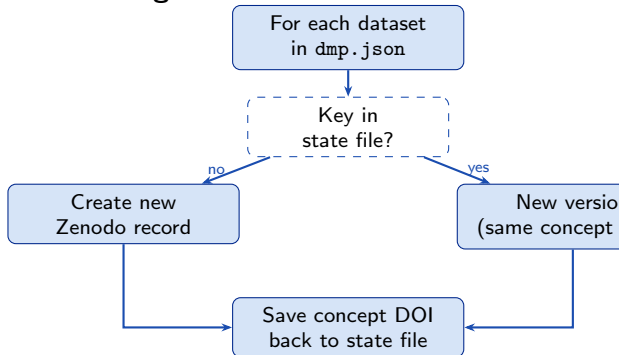
Listing 2: zenodo_state.json

```
{  
  "metabolic-biomarker-cohort-wave1-5":  
    "10.5072/zenodo.499935"  
}
```

Key: `dataset_id.identifier` from the maDMP

Value: Zenodo **concept DOI** — stable across versions

Decision logic on the next release:



Built-in rules and policy

Six built-in FAIR rules:

- ▶ Creator ORCID present *F1, A1*
- ▶ License present & recognized *R1.1*
- ▶ At least one keyword *F2*
- ▶ Description $\geq$ N chars *F2, R1*
- ▶ Access rights stated *A1*
- ▶ Dataset identifier present *F1*

Open:

[github.com/in-silico-institute/
.github/blob/main/.github/policies/
fair-policy.yml](https://github.com/in-silico-institute/.github/blob/main/.github/policies/fair-policy.yml)

Listing 3: fair-policy.yml (in-silico-institute)

```
rules:
- name: Creator ORCID present
  severity: warning
- name: License present and recognized
  severity: error
- name: At least one keyword
  severity: warning
- name: Description length
  severity: warning
  config:
    min_chars: 100
- name: Access rights stated
  severity: error
- name: Dataset identifier present
  severity: error

targets:
- zenodo
```

Institutions add domain-specific rules without modifying the core package.

Listing 4: Example: SNOMED CT rule

```
# myorg_rules/biomedical.py
from fair_publish.rules import RuleResult

class SnomedCtKeyword:
    name = "SNOMED CT keyword present"
    severity = "error"
    fair = "I1, R1.3"

    def __call__(self, dmp, dataset):
        kws = getattr(dataset, "keyword", []) or []
        ok = any("snomed" in k.lower()
                 for k in kws)
        return RuleResult(
            passed=ok, rule=self.name,
            severity=self.severity,
            field="dataset.keyword",
            fair=self.fair,
            message="{} if ok else
                'No SNOMED CT keyword found.',
        )
```

Listing 5: Register in policy.yml

```
rules:
  - name: License present and recognized
    severity: error
  - name: At least one keyword
    severity: error
  - name: Description length
    severity: warning
    config:
      min_chars: 150

extra_rules:
  - module: myorg_rules.biomedical
    class: SnomedCtKeyword
    severity: error

targets:
  - zenodo
```

One policy YAML per institution.
Propagates to all project repos automatically.

What the pipeline provides:

- ▶ Pluggable rule protocol: built-in FAIR rules + institution domain rules
- ▶ One YAML policy at org level, inherited by all projects
- ▶ Per-dataset lifecycle tracking (`zenodo_state.json`):
new record vs. new version decided automatically
- ▶ Two-workflow design: validate on push, deploy on release
- ▶ CLI for local validation and sandbox simulation
- ▶ Low entry barrier: two files per project repo

Current limitations

- ▶ DMP authoring tool is external (assumed to exist)
- ▶ Repos must be institution-owned
- ▶ Branch protection must be configured separately
- ▶ Metadata only — no data file upload yet
- ▶ Policy evolution not automated
- ▶ No error notification mechanism

Live demo org

<https://github.com/in-silico-institute>

Package: `in-silico-institute/fair-publish`

Questions?

`msicilia@uah.es`

`https://github.com/in-silico-institute`